

test_scenario_sets.vr

\$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/scenarios/test_scenario_sets.vr

```
//
// Demonstrate how to use scenarios sets
// i.e. each scenario defined as a different
// extension/instance of the scenario descriptor.
//
#include "scenarios.vrh"
//
// Telegraph good & bad cells
// good - BAD - good - BAD - good
// VPIa   VPIb  VPIa   VPIb  VPIa
//
class telegraph_bad_cells extends atm_cell_scenario {
    integer TELEGRAPH;

    constraint telegraph_scenario {
        length == 5;
        repeated == 0;
        foreach (this.items, i) {
            if (i % 2) this.items[i].hec != 8'h00;
            else      this.items[i].hec == 8'h00;
            if (i > 1) this.items[i].vpi == this.items[i-2].vpi;
            if (i == 1) this.items[i].vpi != this.items[i-1].vpi;
        }
    }
    task new() {
        super.new();
        this.TELEGRAPH = this.define_scenario("Telegraph bad cells", 5);
    }
    task pre_randomize() {
        super.pre_randomize();
        foreach (this.items, i) {
            void = this.items[i].constraint_mode(OFF, "hec_valid");
        }
    }
}
//
```

[illegible]

```

                                var bit                                drop)

{
    scenario.display();
    printf("-----\n");
}
}

program test_gen {
    atm_cell_scenario_gen gen = new("Singleton");
    my_atm_cell_scenario_gen_callbacks cb = new;
    telegraph_bad_cells s1 = new;
    procedural_cells     s2 = new;
    gen.scenario_set.push_back(s1);
    gen.scenario_set.push_back(s2);
    gen.register_callback(cb);
    // Drain the output channel
    fork
    {
        while (1) {
            atm_cell cell = gen.out_chan.get_t();
        }
    }
    join none
    rvm_note(gen.log, "Generating 25 cells or 10 scenarios...");
    gen.stop_after_n_insts = 25;
    gen.stop_after_n_scenarios = 10;
    gen.start_xactor();
    void = gen.notify.wait_for_t(gen.DONE);
    delay(100);
    printf("=====\n");
    // Rnadamly pick scenarios from set now...
    rvm_note(gen.log, "Generating sets in random order...");
    void = gen.select_scenario.constraint_mode(OFF, "round_robin");
    gen.reset_xactor();
    gen.start_xactor();
    void = gen.notify.wait_for_t(gen.DONE);
    delay(100);
    gen.log.report();
}

```